# Lab Handout 2-C

## Byte-Level Receive First In, First Out (FIFO) and Session-5 Pipeline Integration

Capstone — Module 2: Field-Programmable Gate Array (FPGA) Register-Transfer Level (RTL) Design and Signal Ingestion

## 1 Objective

Replace the 16-deep LH1-H `byte_fifo` with a 64-deep version, then verify the combined `uart_rx` → `byte_fifo` pipeline against a recorded Module 1 capture and an overflow stress test. By the end of this handout the project will contain a wider FIFO and two new testbenches.

## 2 Prerequisites

- Lab Handout 2-B (`uart_rx` replaced with the Module 2 generic version and both testbenches passing).

## 3 Background

**Distributed RAM vs. block RAM**

For a depth of 64 and a width of 8, Vivado infers distributed RAM using LUT primitives (about 8 LUTs) rather than consuming a block RAM tile. The Artix-7 100T has only 135 Block Random Access Memory (BRAM) tiles, and they should be reserved for the feature vector memory in Lab Handout 2-F. A distributed-RAM FIFO of this size is essentially free against the 63,400-LUT budget.

A depth of 64 is comfortably larger than the 43-byte Module 1 interceptor forwarding frame. Even if the parser stalls for a full frame time, no bytes are lost. The LH1-H FIFO was 16-deep, which is still functional at the bring-up stage because the simple `frame_detector` drains continuously — but once `pkt_parser` (LH2-D) adds Cyclic Redundancy Check (CRC) computation and reject handling, transient stalls become long enough that 16 bytes is no longer safe.

## 4 Procedure

### 4.1 Step 1 — Save a reference copy of the LH1-H `byte_fifo`

The Module 2 `byte_fifo` keeps the LH1-H entity name and port list, so it is edited in place rather than removed. Before editing, copy the existing file aside for comparison:

1. Outside Vivado, copy `ingest_top.srcs/sources_1/new/byte_fifo.vhd` to a backup location (for example `ingest_top/reference/byte_fifo_lh1h.vhd`). Do not add the backup to the Vivado project.

### 4.2 Step 2 — Edit `byte_fifo.vhd` in place

1. In Vivado's Sources panel, double-click the existing `byte_fifo` entry under Design Sources.
2. Select all (`Ctrl+A`), delete the LH1-H contents, and paste the listing below verbatim. Save (`Ctrl+S`).

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity byte_fifo is
    generic ( DEPTH : integer := 64 );
    port (
        clk     : in  std_logic;
        rst_n   : in  std_logic;
        wr_en   : in  std_logic;
        wr_data : in  std_logic_vector(7 downto 0);
        rd_en   : in  std_logic;
        rd_data : out std_logic_vector(7 downto 0);
        empty   : out std_logic;
        full    : out std_logic
    );
end byte_fifo;

architecture rtl of byte_fifo is
    type mem_t is array (0 to DEPTH - 1) of std_logic_vector(7 downto 0);
    signal mem   : mem_t := (others => (others => '0'));
    signal w_ptr : integer range 0 to DEPTH - 1 := 0;
    signal r_ptr : integer range 0 to DEPTH - 1 := 0;
    signal count : integer range 0 to DEPTH     := 0;
begin
    full  <= '1' when count = DEPTH else '0';
    empty <= '1' when count = 0     else '0';

    process (clk)
        variable do_wr, do_rd : boolean;
    begin
        if rising_edge(clk) then
            if rst_n = '0' then
                w_ptr <= 0; r_ptr <= 0; count <= 0;
            else
                do_wr := (wr_en = '1') and (count < DEPTH);
                do_rd := (rd_en = '1') and (count > 0);
                if do_wr then
                    mem(w_ptr) <= wr_data;
                    w_ptr <= (w_ptr + 1) mod DEPTH;
                end if;
                if do_rd then
                    r_ptr <= (r_ptr + 1) mod DEPTH;
                end if;
                if    do_wr and not do_rd then count <= count + 1;
                elsif do_rd and not do_wr then count <= count - 1;
                end if;
```

```
            end if;
            rd_data <= mem(r_ptr);     -- registered read; valid 1 cycle
                after rd_en
        end if;
    end process;
end rtl;
```

Listing 1: `byte_fifo.vhd` (Module 2) — 64-entry × 8-bit synchronous FIFO with registered read output. Infers distributed RAM on the Artix-7. Port list matches LH1-H exactly.

### 4.3 Step 3 — Re-synthesize and confirm hardware still runs

1. **Confirm `ingest_top` is set as the Design Sources top.** In the Sources panel's *Design Sources* tab, `ingest_top` must appear in **bold**. If a testbench or other entity is bold instead (which can happen after Module 2 simulation runs), right-click `ingest_top` → **Set as Top** before continuing.

2. Run synthesis. Expect zero errors. The `u_fifo` red "!" marker should clear.

3. Re-program the board. The Beacon ID display should still update — the wider FIFO is functionally equivalent to the LH1-H 16-deep version for a continuously-draining frame detector.

**[Figure placeholder]**

`LH2C_session5_datapath.png`

*Session 5 datapath after integration: synchronizer → `uart_rx` → `byte_fifo`, with occupancy LEDs routed to `LED[2]` (empty) and `LED[3]` (full) in `ingest_top`.*

Figure 1: Session 5 datapath after integration: synchronizer → `uart_rx` → `byte_fifo`, with occupancy LEDs routed to `LED[2]` (empty) and `LED[3]` (full) in `ingest_top`.

### 4.4 Step 4 — Unit testbench

Add a simulation-source file `tb_byte_fifo`. Replace its contents with the listing below. The testbench writes 16 bytes, reads them back, and asserts the read data matches the write data in order.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity tb_byte_fifo is
```

```
end tb_byte_fifo;

architecture sim of tb_byte_fifo is
    constant CLK_PERIOD : time := 10 ns;

    signal clk     : std_logic := '0';
    signal rst_n   : std_logic := '0';
    signal wr_en   : std_logic := '0';
    signal wr_data : std_logic_vector(7 downto 0) := (others => '0');
    signal rd_en   : std_logic := '0';
    signal rd_data : std_logic_vector(7 downto 0);
    signal empty   : std_logic;
    signal full    : std_logic;

    signal mismatch_cnt : integer := 0;
begin
    clk <= not clk after CLK_PERIOD / 2;

    dut : entity work.byte_fifo
        port map (clk => clk, rst_n => rst_n,
                  wr_en => wr_en, wr_data => wr_data,
                  rd_en => rd_en, rd_data => rd_data,
                  empty => empty, full => full);

    process
        variable expected : std_logic_vector(7 downto 0);
    begin
        wait for 100 ns;
        rst_n <= '1';
        wait until rising_edge(clk);

        -- Write 16 ascending bytes 0x10..0x1F.
        for i in 0 to 15 loop
            wr_data <= std_logic_vector(to_unsigned(16#10# + i, 8));
            wr_en   <= '1';
            wait until rising_edge(clk);
        end loop;
        wr_en <= '0';
        wait until rising_edge(clk);

        assert empty = '0' report "FIFO should not be empty after 16 writes" severity failure;
        assert full  = '0' report "FIFO should not be full at depth 16 of 64" severity failure;

        -- Read 16 bytes and compare. Account for 1-cycle read latency by
        -- comparing rd_data on the cycle AFTER each rd_en pulse.
        for i in 0 to 15 loop
            rd_en <= '1';
            wait until rising_edge(clk);
            rd_en <= '0';
            wait until rising_edge(clk);                -- registered output appears now
            expected := std_logic_vector(to_unsigned(16#10# + i, 8));
```

```
            if rd_data /= expected then
                mismatch_cnt <= mismatch_cnt + 1;
            end if;
        end loop;

        assert mismatch_cnt = 0
            report "tb_byte_fifo: " & integer'image(mismatch_cnt) & " byte
                mismatch(es)"
            severity failure;
        assert empty = '1' report "FIFO should be empty after 16 reads"
           severity failure;
        report "tb_byte_fifo PASS" severity note;
        std.env.finish;
    end process;
end sim;
```

Listing 2: `tb_byte_fifo.vhd` — write-then-read unit test. Verifies one-cycle read latency and pointer wrap.

Set as simulation top, launch behavioral simulation. The default `1us` runtime stops short of the final assertion; in the Tcl console, type `run 5 us` (once) and press Enter. The Tcl console should print `tb_byte_fifo PASS` within about 1 us of simulated time. Total wall-clock cost is a few seconds.

### 4.5 Step 5 — Overflow stress testbench

Add a second simulation-source file `tb_byte_fifo_overflow`. Replace its contents with the listing below.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity tb_byte_fifo_overflow is
end tb_byte_fifo_overflow;

architecture sim of tb_byte_fifo_overflow is
    constant CLK_PERIOD : time := 10 ns;

    signal clk     : std_logic := '0';
    signal rst_n   : std_logic := '0';
    signal wr_en   : std_logic := '0';
    signal wr_data : std_logic_vector(7 downto 0) := (others => '0');
    signal rd_en   : std_logic := '0';
    signal rd_data : std_logic_vector(7 downto 0);
    signal empty   : std_logic;
    signal full    : std_logic;

    signal full_seen_at : integer := -1;
begin
    clk <= not clk after CLK_PERIOD / 2;

    dut : entity work.byte_fifo
        port map (clk => clk, rst_n => rst_n,
```

```vhdl
                    wr_en => wr_en, wr_data => wr_data,
                    rd_en => rd_en, rd_data => rd_data,
                    empty => empty, full => full);

    process
    begin
        wait for 100 ns;
        rst_n <= '1';
        wait until rising_edge(clk);

        for i in 0 to 64 loop
            wr_data <= std_logic_vector(to_unsigned(i, 8));
            wr_en   <= '1';
            wait until rising_edge(clk);
            if full = '1' and full_seen_at = -1 then
                full_seen_at <= i;
            end if;
        end loop;
        wr_en <= '0';
        wait until rising_edge(clk);

        assert full = '1'
            report "FIFO should still be full after 65 writes" severity failure;
        assert full_seen_at = 64
            report "full should first assert when count reaches 64, observed at "
                & integer'image(full_seen_at) severity failure;

        -- Reset and confirm the pointers/count return to a clean state.
        rst_n <= '0';
        wait until rising_edge(clk);
        rst_n <= '1';
        wait until rising_edge(clk);
        wait until rising_edge(clk);
        assert empty = '1' report "FIFO should be empty after reset" severity failure;
        assert full  = '0' report "FIFO should not be full after reset" severity failure;

        report "tb_byte_fifo_overflow PASS" severity note;
        std.env.finish;
    end process;
end sim;
```

Listing 3: `tb_byte_fifo_overflow.vhd` — writes 65 bytes with no reads. Asserts `full` latches at depth and pointers do not corrupt.

Set as simulation top, launch behavioral simulation, then in the Tcl console type `run 5 us` (once) and press Enter. Confirm `tb_byte_fifo_overflow PASS` prints in the Tcl console; total simulated time is well under 2 us.

## 5 Verification Checklist

- ☐ `byte_fifo.vhd` edited in place to the Module 2 (depth 64) version; LH1-H copy saved aside as a reference.
- ☐ `tb_byte_fifo` prints `PASS`.
- ☐ `tb_byte_fifo_overflow` prints `PASS` with `full` first asserting on the 64th write.
- ☐ Re-synthesis succeeds and the seven-segment display still updates on hardware.
- ☐ Post-synthesis utilization shows 0 BRAM tiles consumed by the FIFO (distributed RAM inferred).

**Lab Handout 2-C Deliverable**

1. Module 2 `byte_fifo.vhd`, `tb_byte_fifo.vhd`, and `tb_byte_fifo_overflow.vhd`.
2. Tcl console transcripts showing both testbenches `PASS`.
3. Post-synthesis utilization screenshot annotating FIFO LUT cost and confirming BRAM = 0.

# Glossary

| | | |
|---|---|---|
| **BRAM** | Block Random Access Memory | 1, 7 |
| **CRC** | Cyclic Redundancy Check | 1 |
| **FIFO** | First In, First Out | 1, 3, 7 |
| **FPGA** | Field-Programmable Gate Array | 1 |
| **RTL** | Register-Transfer Level | 1 |